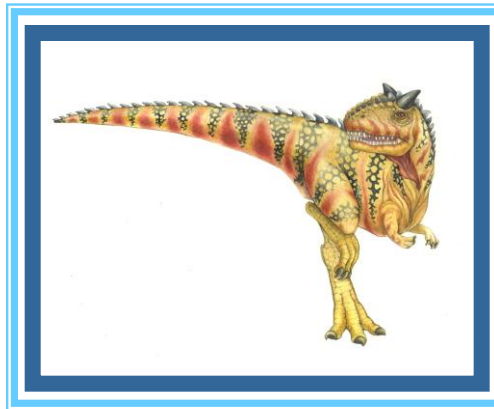
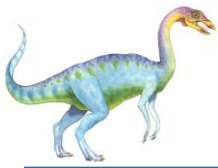


Chapter 4: Threads & Concurrency





Outline

- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues
- Operating System Examples





Objectives

- Identify the basic components of a thread, and contrast threads and processes
- Describe the benefits and challenges of designing multithreaded applications
- Illustrate different approaches to implicit threading including thread pools, fork-join, and Grand Central Dispatch
- Describe how the Windows and Linux operating systems represent threads
- Designing multithreaded applications using the Pthreads, Java, and Windows threading APIs





Motivation

- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
 - Update display
 - Fetch data
 - Spell checking
 - Answer a network request
- Process creation is heavy-weight while thread creation is light-weight
- Can simplify code, increase efficiency
- Kernels are generally multithreaded





Problems with Processes

- Process creation is heavyweight (i.e. slow)
 - Space must be allocated for the new process
 - `fork()` copies all state of the parent to the child
- IPC mechanisms are cumbersome
 - Difficult to use fine-grained synchronization
 - Message passing is slow
 - ▶ Each message may have to go through the kernel

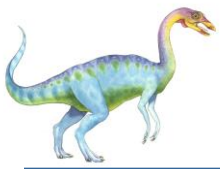




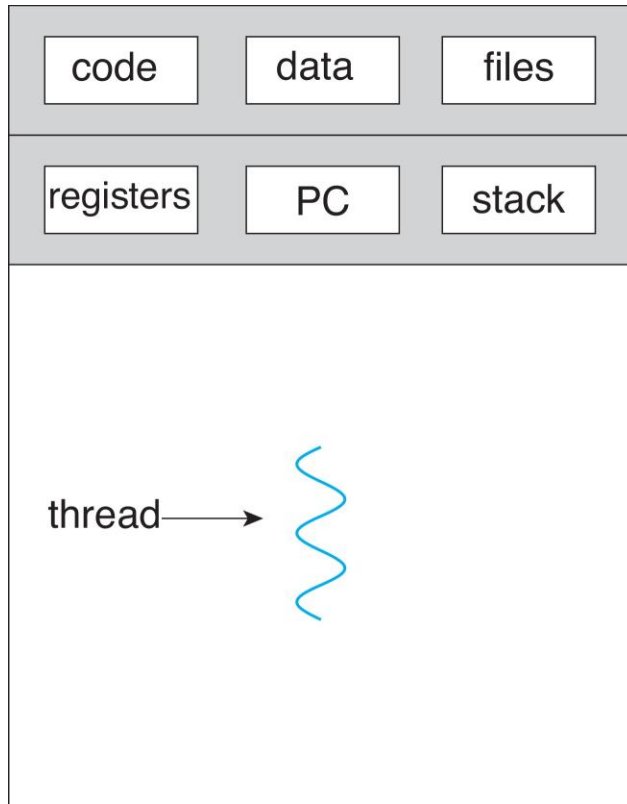
Threads

- Light-weight processes that share the same memory and state space
- Every process has at least one thread
- Benefits:
 - Resource sharing, no need for IPC
 - Economy: faster to create, faster to context switch
 - Scalability: simple to take advantage of multi-core CPUs

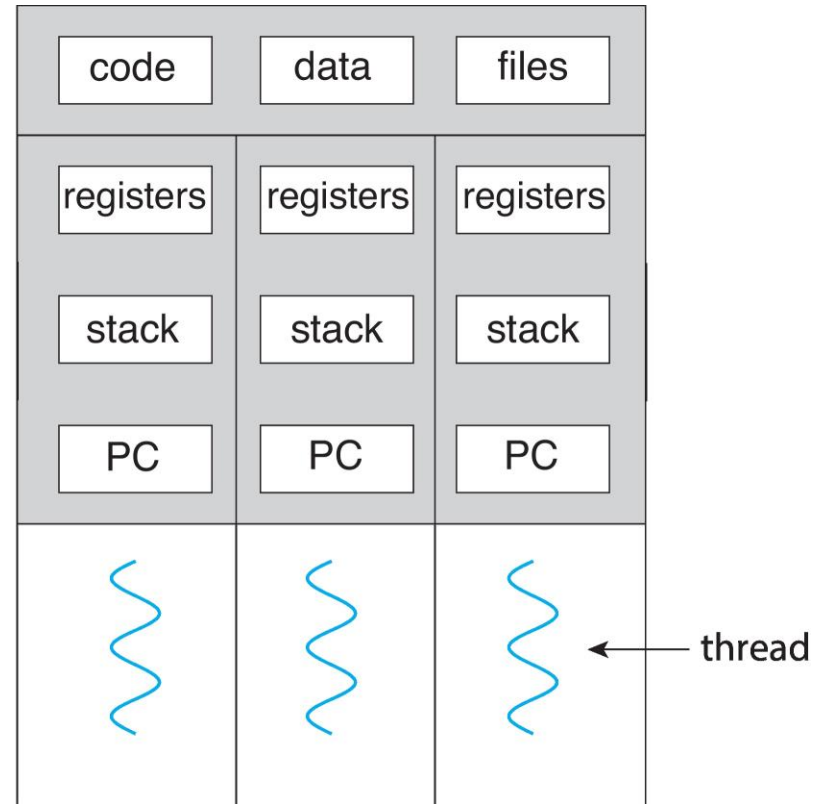




Single and Multithreaded Processes

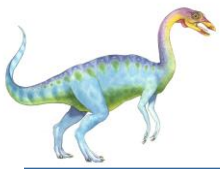


single-threaded process

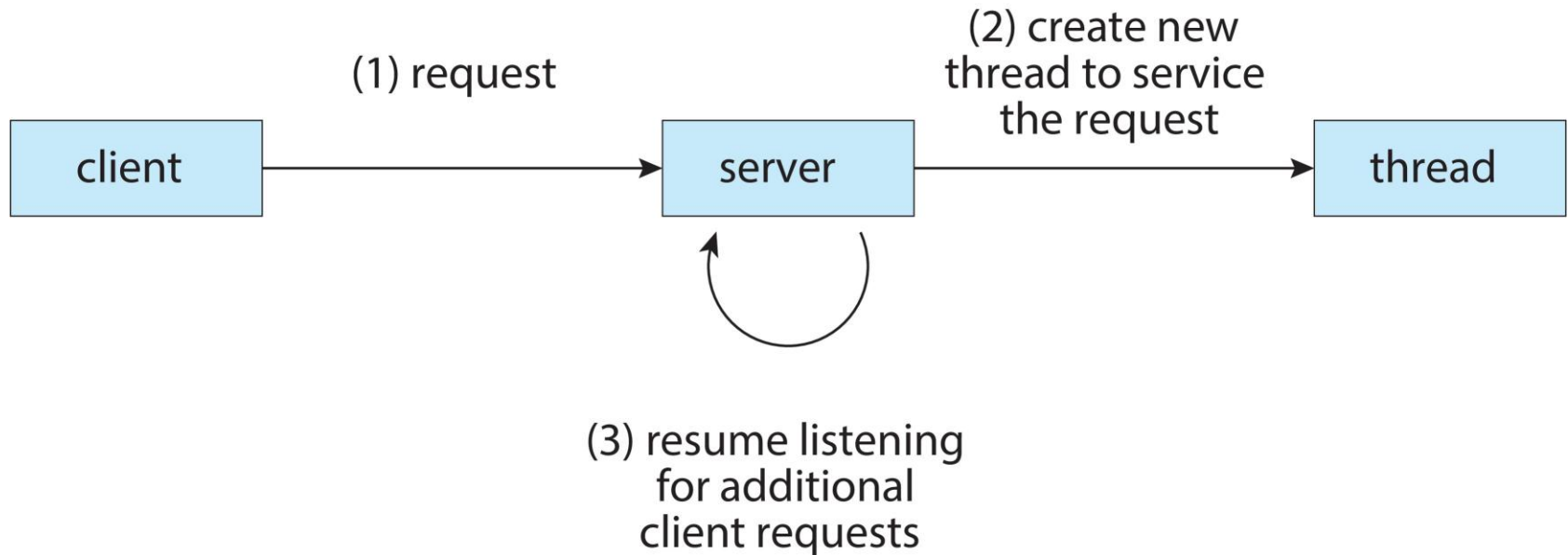


multithreaded process





Multithreaded Server Architecture





Benefits

- **Responsiveness** – may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing** – threads share resources of process, easier than shared memory or message passing
- **Economy** – cheaper than process creation, thread switching lower overhead than context switching
- **Scalability** – process can take advantage of multicore architectures





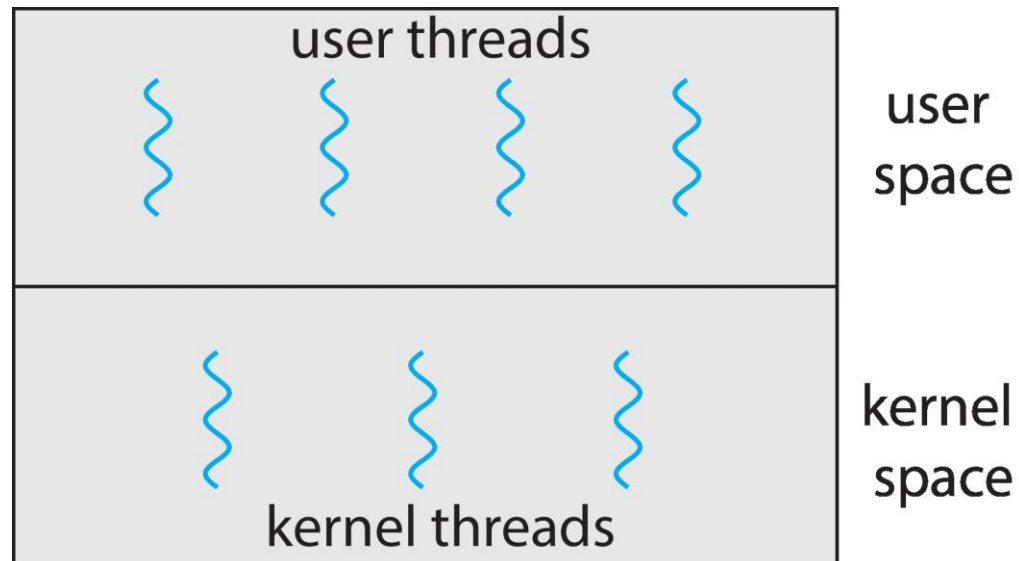
User Threads and Kernel Threads

- **User threads** - management done by user-level threads library
- Three primary thread libraries:
 - POSIX **Pthreads**
 - Windows threads
 - Java threads
- **Kernel threads** - Supported by the Kernel
- Examples – virtually all general-purpose operating systems, including:
 - Windows
 - Linux
 - Mac OS X
 - iOS
 - Android





User and Kernel Threads





Multithreading Models

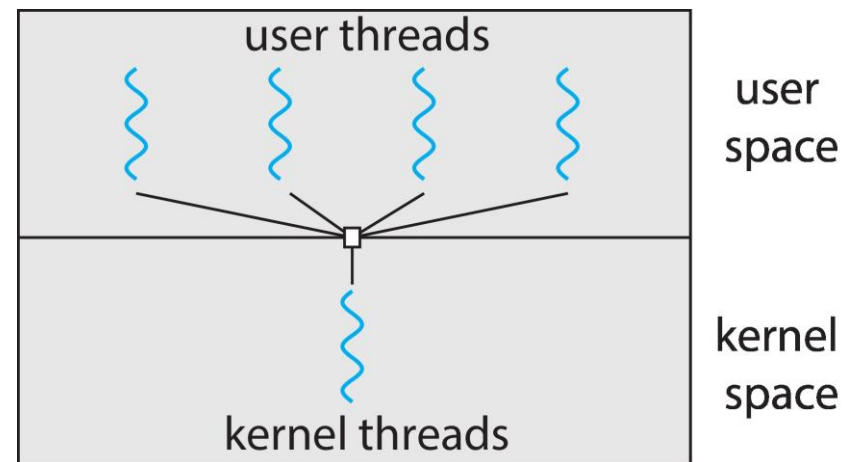
- Many-to-One
- One-to-One
- Many-to-Many





Many-to-One

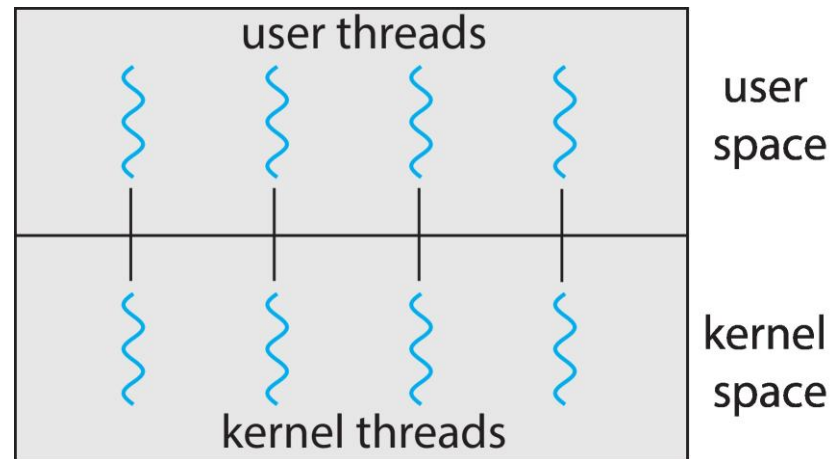
- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
 - **Solaris Green Threads**
 - **GNU Portable Threads**





One-to-One

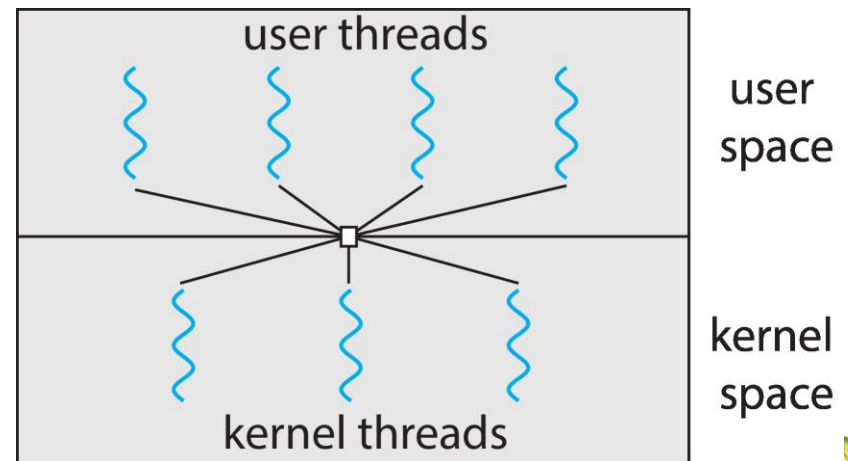
- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
 - Windows
 - Linux





Many-to-Many Model

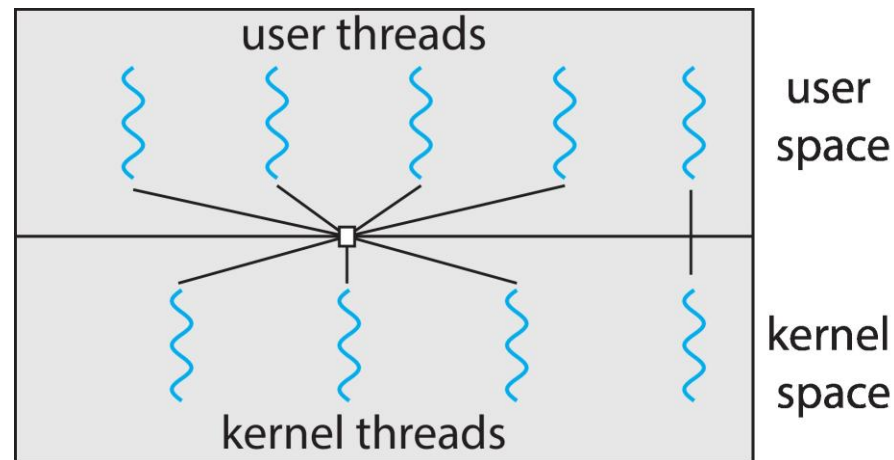
- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads.
- Allows to create sufficient number of kernel threads
 - Threads can run in parallel
- With increasing number of processing cores, limiting number of kernel threads has become less important.





Two-level Model

- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread.
- The key idea is that most user threads are mapped to kernel threads dynamically, but some user threads can also be permanently bound to specific kernel threads when needed.
- Giving better performance and more control for important threads.

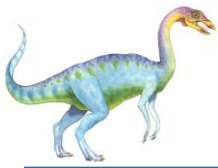




Thread Libraries

- **Thread library** provides programmer with API for creating and managing threads
- Two primary ways of implementing
 - Library entirely in user space
 - Kernel-level library supported by the OS





Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- ***Specification***, not ***implementation***
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Linux & Mac OS X)





Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    /* set the default attributes of the thread */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid, NULL);

    printf("sum = %d\n", sum);
}
```





Pthreads Example (Cont.)

```
/* The thread will execute in this function */  
void *runner(void *param)  
{  
    int i, upper = atoi(param);  
    sum = 0;  
  
    for (i = 1; i <= upper; i++)  
        sum += i;  
  
    pthread_exit(0);  
}
```





Thread Pools

- Create a number of threads in a pool where they await work
- A **thread pool** is a collection of pre-created worker threads that are kept ready to execute tasks. Instead of creating a new thread for each request, tasks are assigned to an available thread in the pool.
- Advantages:
 - Usually slightly faster to service a request with an existing thread than create a new thread
 - Allows the number of threads in the application(s) to be bound to the size of the pool
 - Separating task to be performed from mechanics of creating task allows different strategies for running task
 - ▶ i.e, Tasks could be scheduled to run periodically





Thread Pools

- Windows API supports thread pools. Provides built-in thread pool support through the **Windows Thread Pool API**.
- **Linux:** POSIX threads (pthread) **do not provide a built-in thread pool API**. Thread pools are typically implemented by applications or libraries using pthread threads.
- Internally, Linux also uses **kernel worker threads** to perform background kernel tasks.





Threading Issues

- Semantics of **fork()** and **exec()** system calls
- Signal handling
 - Synchronous and asynchronous
- Thread cancellation of target thread
 - Asynchronous or deferred
- Thread-local storage
- Scheduler Activations





Semantics of `fork()` and `exec()`

- Does `fork()` duplicate only the calling thread or all threads?
 - Some UNIXes have two versions of `fork`
- `exec()` usually works as normal – replace the running process including all threads





Semantics of fork() and exec()

1. The fork() and exec() System Calls

- The fork() system call is used to create a separate, duplicate process.
- If one thread in a program calls fork(), does the new process duplicate all threads, or is the new process single-threaded?
- Some UNIX systems have chosen to have two versions of fork(),
 - one that duplicates all threads and
 - another that duplicates only the thread that invoked the fork() system call.
- If a thread invokes the exec() system call, the program specified in the parameter to exec() will replace the entire process—including all threads.





Semantics of fork() and exec()

1. The fork() and exec() System Calls...

- If exec() is called immediately after forking, then duplicating all threads is unnecessary, as the program specified in the parameters to exec() will replace the process.
- In this instance, duplicating only the calling thread is appropriate.
- If, however, the separate process does not call exec() after forking, the separate process should duplicate all threads.





Signal Handling

- **Signals** are used in UNIX systems to notify a process that a particular event has occurred.
- A **signal handler** is used to process signals
 1. Signal is generated by particular event
 2. Signal is delivered to a process
 3. Signal is handled by one of two signal handlers:
 1. default
 2. user-defined
- Every signal has **default handler** that kernel runs when handling signal
 - **User-defined signal handler** can override default
 - For single-threaded, signal delivered to process





Signal Handling (Cont.)

- Where should a signal be delivered for multi-threaded?
 - ▶ Deliver the signal to the thread to which the signal applies
 - ▶ Deliver the signal to every thread in the process
 - ▶ Deliver the signal to certain threads in the process
 - ▶ Assign a specific thread to receive all signals for the process





Thread Cancellation

- Terminating a thread before it has finished
- Thread to be canceled is **target thread**
- Two general approaches:
 - **Asynchronous cancellation** terminates the target thread immediately
 - **Deferred cancellation** allows the target thread to periodically check if it should be cancelled. (Default in POSIX)
- Pthread code to create and cancel a thread:

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

. . .

/* cancel the thread */
pthread_cancel(tid);

/* wait for the thread to terminate */
pthread_join(tid, NULL);
```





Thread-Local Storage

- **Thread-local storage (TLS)** allows each thread to have its own copy of data
- TLS variables are **not shared among threads**. They remain available throughout the lifetime of the thread and can be accessed across multiple function calls within that thread.
- Useful when you do not have control over the thread creation process (i.e., when using a thread pool)
- TLS is especially useful in thread pools and multithreaded applications where each thread needs to maintain its own independent state without interfering with other threads.
- Different from local variables
 - Local variables visible only during single function invocation
 - TLS visible across function invocations
- Similar to **static** data
 - TLS is unique to each thread



End of Chapter 4

